OPERATING SYSTEMS

Unit-3: Memory Management

Comprehensive Study Notes

What is Memory Management?

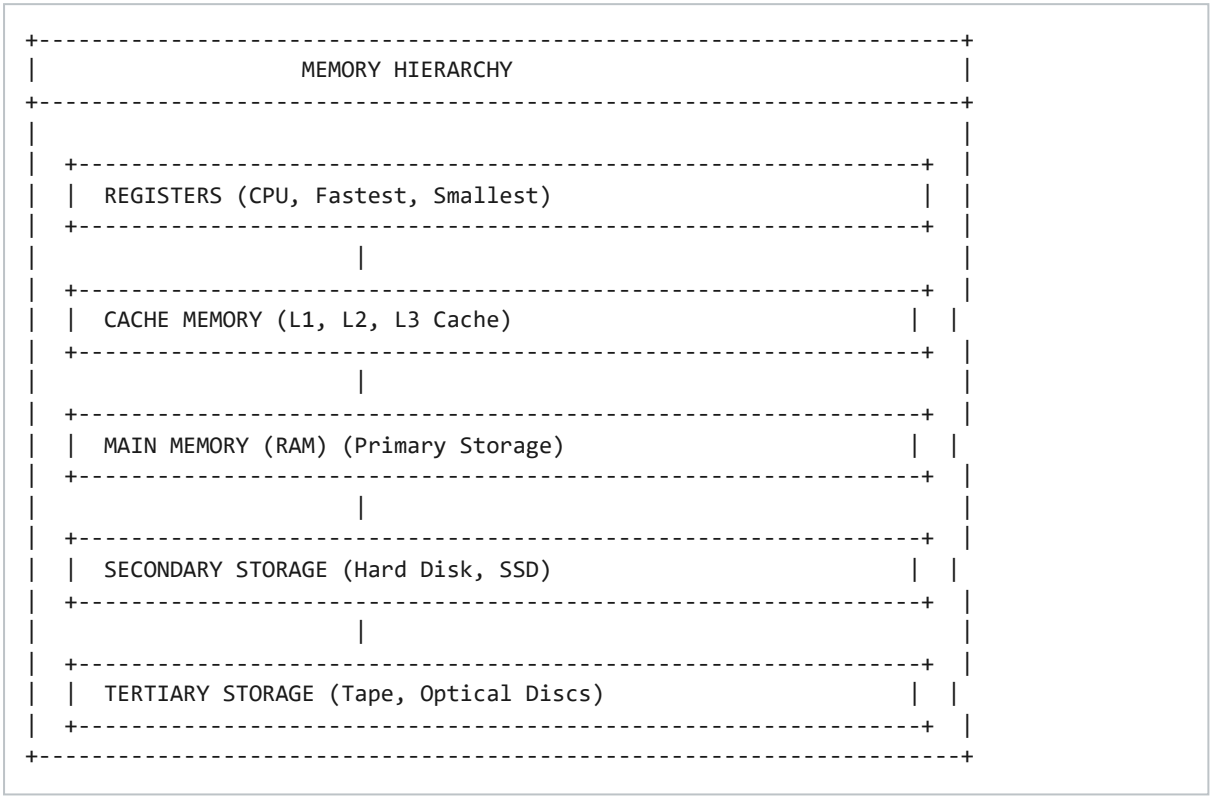
Memory Management is the function of the operating system that manages the computer's **primary memory** (RAM). It involves:

- **Allocating** memory to processes
- **Deallocating** memory when processes finish
- **Protecting** memory from unauthorized access
- **Managing** virtual memory and swapping

Why is Memory Management Needed?

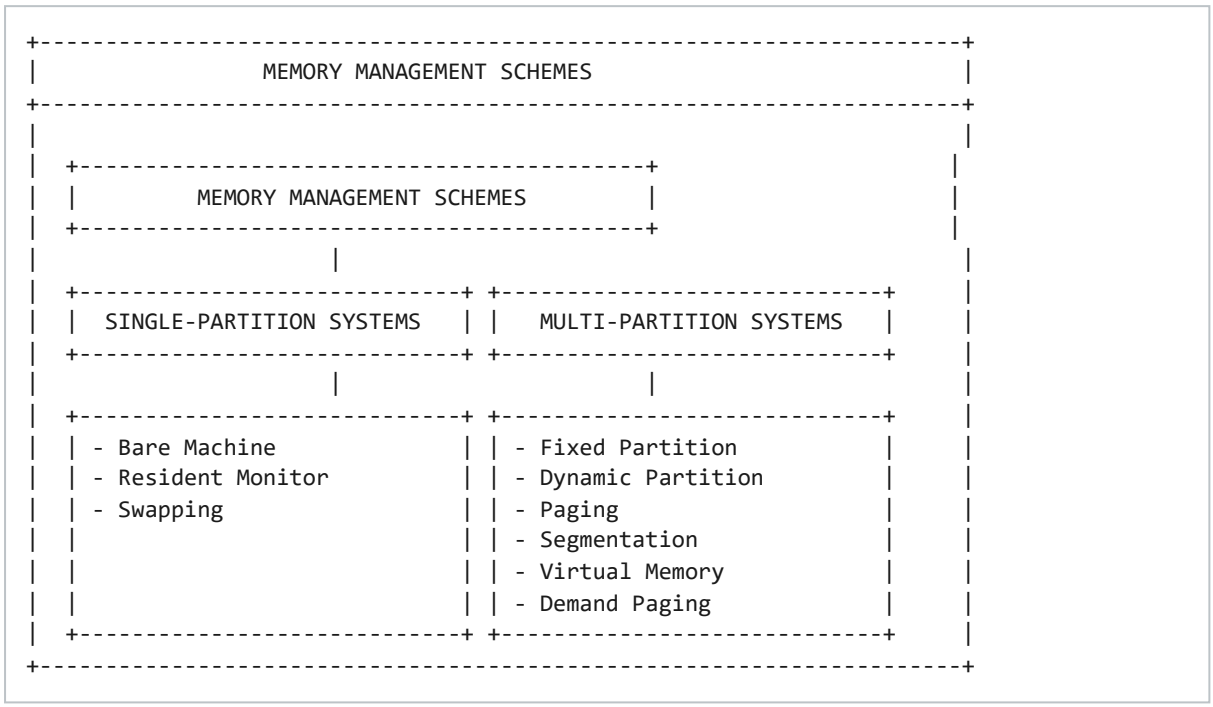
```
+---------------------------------------------------------------------+
|                 REASONS FOR MEMORY MANAGEMENT                       |
+---------------------------------------------------------------------+
|                                                                     |
| 1. MULTIPROGRAMMING                                                 |
|    - Multiple programs in memory simultaneously                     |
|    - Need to allocate memory efficiently                            |
|                                                                     |
| 2. MEMORY PROTECTION                                                |
|    - Prevent processes from accessing other processes' memory       |
|    - Prevent processes from accessing OS memory                     |
|                                                                     |
| 3. EFFICIENT UTILIZATION                                            |
|    - Use available memory efficiently                               |
|    - Minimize memory wastage                                        |
|                                                                     |
| 4. VIRTUAL MEMORY                                                   |
|    - Run programs larger than physical memory                       |
|    - Use disk as extension of memory                                |
|                                                                     |
| 5. FASTER EXECUTION                                                 |
|    - Keep frequently used data in memory                            |
|    - Minimize disk access                                           |
+---------------------------------------------------------------------+
```

Memory Hierarchy



Level	Type	Speed	Size	Cost
1	Registers	Fastest	Smallest (bytes)	Highest
2	Cache (L1/L2/L3)	Very Fast	KB - MB	High
3	RAM (Main Memory)	Fast	GB	Medium
4	SSD / Hard Disk	Slow	TB	Low
5	Tape / Optical	Slowest	Largest	Lowest

Memory Management Schemes



Address Binding

Address Binding is the mapping of program addresses (logical/virtual) to actual physical memory locations. It can occur at different stages:

Binding Type	Stage	Description
Compile Time	Compilation	Absolute code generated; if starting address changes, code must be recompiled
Load Time	Loading	Compile in relocatable format; binding delayed until load time
Execution Time	Runtime	Binding done during execution; requires hardware (MMU) support; allows process relocation during execution

Logical vs Physical Address

Logical Address	Physical Address
Generated by CPU (virtual address)	Actual location in memory (RAM)
User program sees logical address	Hardware sees physical address
Relative to program's address space	Absolute address in memory
Mapped to physical via MMU	Directly accessed by memory unit

MMU (Memory Management Unit)

The **MMU** is hardware that maps logical addresses to physical addresses at runtime. The simplest scheme uses a **relocation register**: Physical Address = Logical Address + Relocation Register value.

Dynamic Loading & Linking

Dynamic Loading

A routine is not loaded into memory until it is called. All routines are kept on disk in relocatable format. **Advantage:** Better memory utilization – unused routines are never loaded.

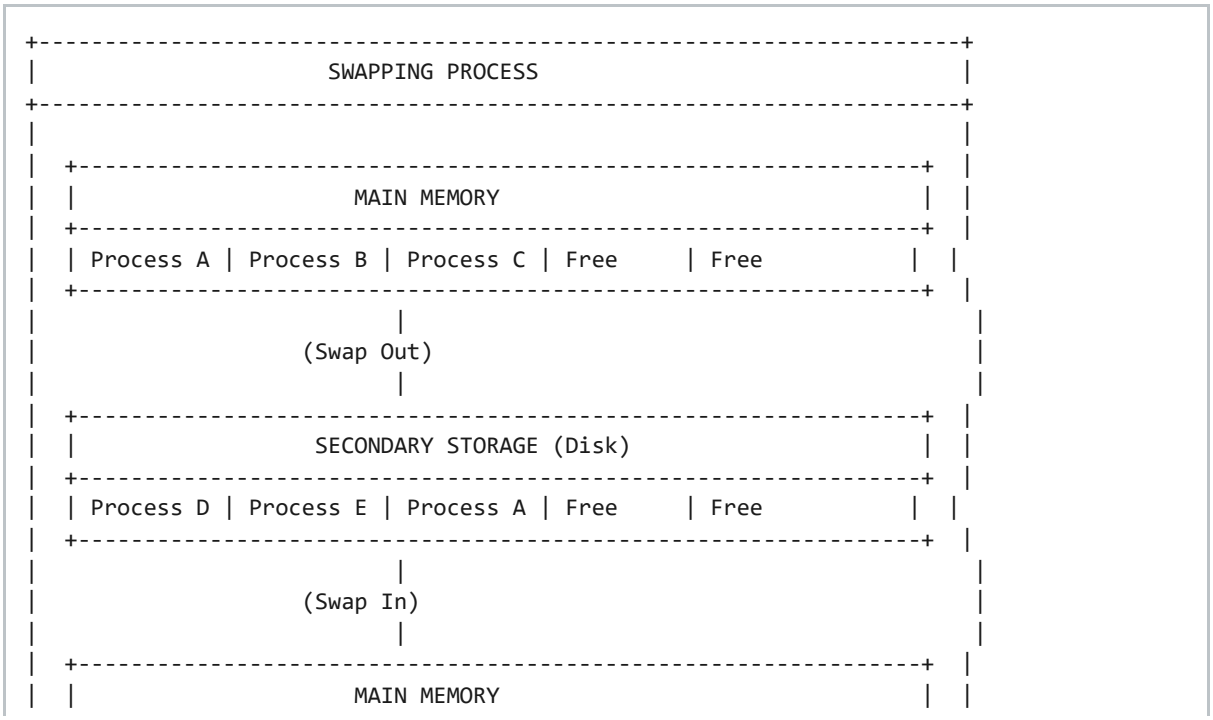
Dynamic Linking

Linking is postponed until execution time. **Shared libraries** (DLLs) are linked when a program references them. Multiple programs can share the same library code in memory.

Swapping

Swapping moves processes temporarily between main memory and secondary storage (disk).

- **Swap Out:** Move process from memory to disk
- **Swap In:** Move process from disk to memory
- **Roll Out / Roll In:** Variants for priority-based scheduling



```
|  +-------------------------------------------------------------+  |
|  | Process D | Process B | Process A | Free     | Free           |  |
|  +-------------------------------------------------------------+  |
+-----------------------------------------------------------------+
```

Partitioning

Partitioning divides main memory into partitions (blocks) that are allocated to processes.

Fixed (Static) Partitioning

```
+-------------------------------------------------------------+
| Partition 1 | Partition 2 | Partition 3 | Partition 4 | Free    |
| (4 MB)      | (6 MB)      | (8 MB)      | (10 MB)     |         |
+-------------------------------------------------------------+
```

- Memory divided into fixed size partitions
- Partitions cannot change size
- Each partition holds one process
- **Problem:** Internal fragmentation

Dynamic (Variable) Partitioning

```
+-------------------------------------------------------------+
| Process A | Process B | Free  | Process C | Free             |
| (5 MB)    | (3 MB)    | (2 MB)| (4 MB)    | (6 MB)           |
+-------------------------------------------------------------+
```

- Partitions created dynamically
- Size of partition = Size of process
- **Problem:** External fragmentation

Allocation Strategies

Strategy	Description	Example
First Fit	Allocate first hole that fits	Process 5MB → First hole ≥ 5MB
Best Fit	Allocate smallest hole that fits	Process 5MB → Smallest hole ≥ 5MB
Worst Fit	Allocate largest hole	Process 5MB → Largest hole available

Next Fit	Continue from last allocation	Start search where last left off

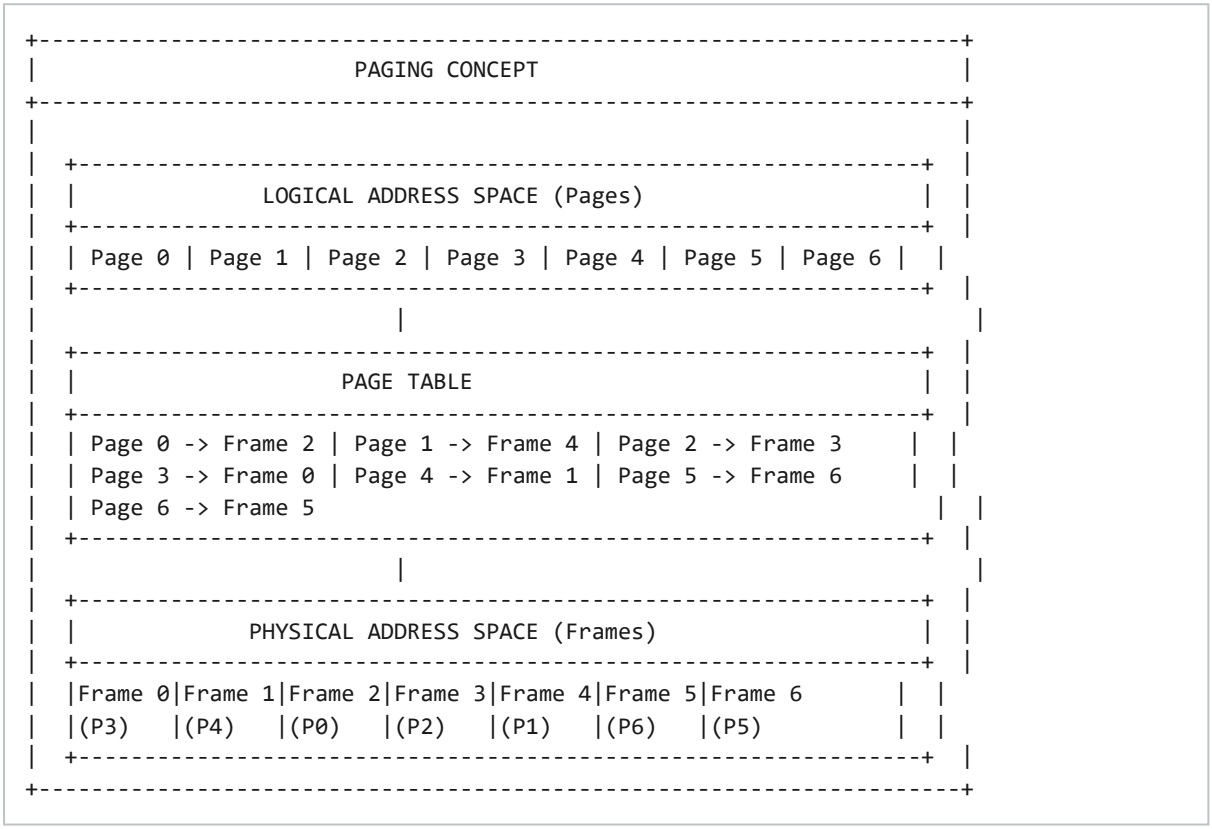
Fragmentation

Internal Fragmentation	External Fragmentation
Unused space inside a partition	Total free memory exists but in small chunks
Occurs in Fixed Partitioning	Occurs in Dynamic Partitioning
Example: 6MB partition, 5MB process → 1MB wasted	Example: 10MB free but split into 2+3+5MB chunks
Solution for External Fragmentation: Compaction – move processes to create one large free block (expensive operation)	

Paging

Paging eliminates the need for contiguous allocation. It divides:

- **Physical Memory** into fixed-size blocks called **Frames**
- **Logical Memory** into same-size blocks called **Pages**



Address Translation in Paging

Logical Address: [Page Number (p) | Offset (d)]
Physical Address: Frame Number × Frame Size + Offset

Example:
Page Size = 4KB (4096 bytes)
Logical Address = 15000
Page Number = 15000 / 4096 = 3
Offset = 15000 % 4096 = 2712
Page Table: Page 3 → Frame 7
Physical Address = 7 × 4096 + 2712 = 31384

Page Table Structure

Frame Number	Present Bit	Modified Bit	Protection Bits
Physical frame location	Page in memory (1) or on disk (0)	Page has been modified (dirty)	Read/Write/Execute permissions

Types of Page Tables

- **Single-level Page Table**
- **Two-level (Hierarchical) Page Table**
- **Hashed Page Table**
- **Inverted Page Table**

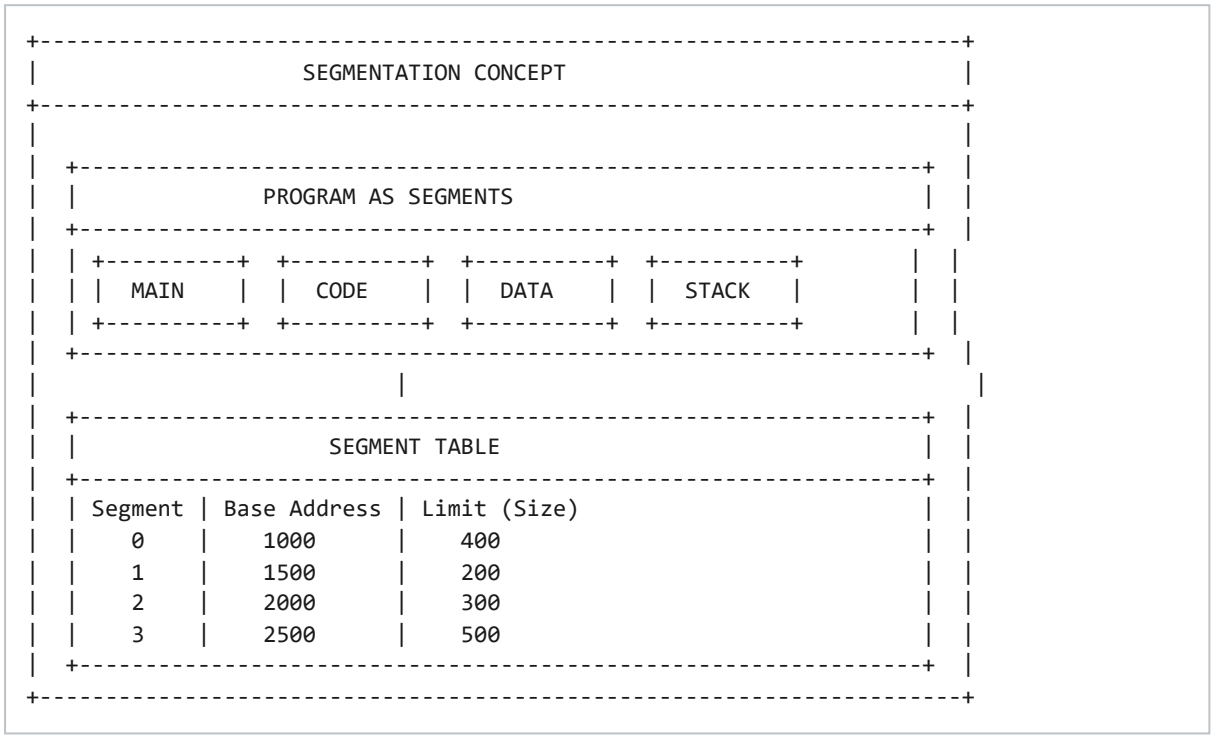
TLB (Translation Lookaside Buffer)

TLB is a fast cache that stores recent page number → frame number mappings. It speeds up address translation.

Effective Access Time (EAT):
EAT = (1 - p) × Memory Access Time + p × Page Fault Time
where p = page fault rate

Segmentation

Segmentation divides a program into **logical segments** (code, data, stack, heap).



Address Translation in Segmentation

Logical Address = (Segment Number, Offset)
If Offset < Limit: Physical Address = Base + Offset
If Offset ≥ Limit: Segmentation Fault (illegal access)

Segmentation vs Paging

Aspect	Segmentation	Paging
Division	Logical units (program structure)	Fixed-size blocks
Size	Variable	Fixed
User View	User sees segments	User sees linear address space

Fragmentation	External	Internal
Memory	Non-contiguous	Non-contiguous
Protection	Per segment	Per page
Overhead	More complex	Simpler

Segmentation with Paging

Each segment is divided into pages. The segment table entry points to a page table for that segment. Combines benefits of both: logical views (segmentation) with efficient allocation (paging).

Virtual Memory

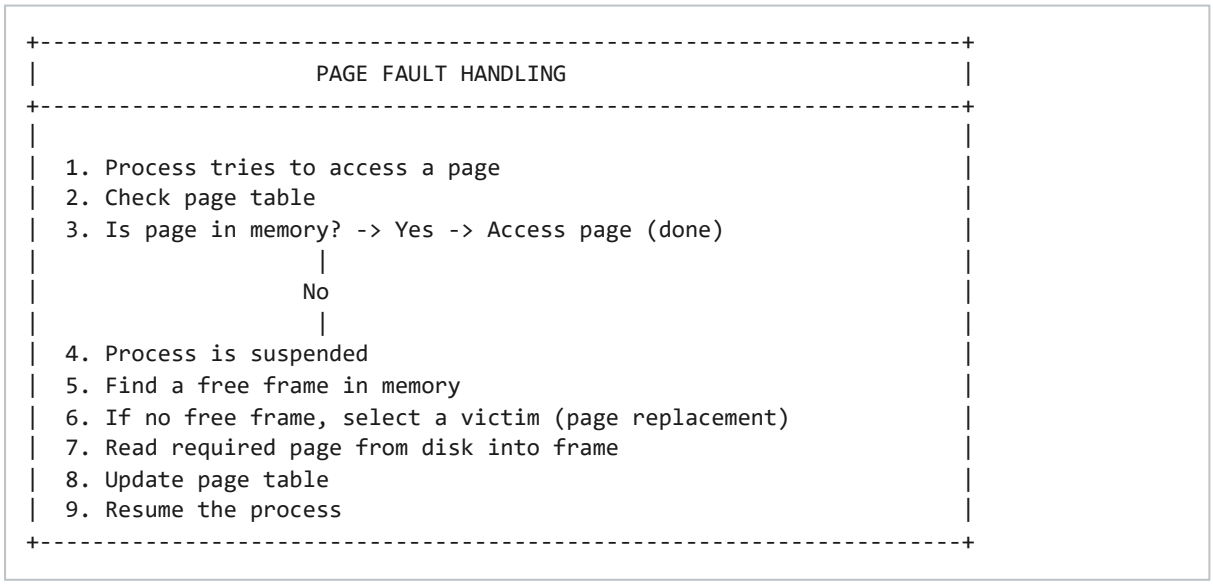
Virtual Memory creates an illusion of a very large memory by using disk as an extension of RAM. Programs larger than physical memory can run.

Advantage	Description
Large Address Space	Programs larger than physical memory can run
Protection	Each process has separate address space
Sharing	Code and data can be shared between processes
Efficient Memory Use	Only necessary pages loaded in memory
Multiprogramming	More processes can be loaded simultaneously

Demand Paging

Pages are **loaded into memory only when needed** (on demand), rather than loading the entire program at once. When a process accesses a page not in memory, a **page fault** occurs.

Page Fault Handling



Copy-on-Write

When a process is forked (using `fork()`), the parent and child initially share the same pages. Pages are only copied when one of them writes to a page. This saves memory and time by avoiding unnecessary copies of pages that are only read.

Page Replacement Algorithms

When no free frame is available, a **victim page** must be selected for replacement. The goal is to minimize the **page fault rate**.

1. FIFO (First-In-First-Out)

Replace the **oldest page** in memory. Simple to implement but suffers from **Belady's Anomaly** (more frames can cause more page faults).

Reference	1	2	3	4	1	2	5	1	2	3	4	5
Frame 1	1	1	1	4	4	4	5	5	5	5	5	5
Frame 2		2	2	2	1	1	1	1	1	3	3	3
Frame 3			3	3	3	2	2	2	2	2	4	4
Fault?	Y	Y	Y	Y	Y	Y	Y	N	N	Y	Y	Y

Total Page Faults: 9 (with 3 frames)

2. Optimal Page Replacement (OPT)

Replace the page that **will not be used for the longest time** in the future. Theoretically optimal but cannot be implemented in practice. Used as a **benchmark**.

Reference	1	2	3	4	1	2	5	1	2	3	4	5
Frame 1	1	1	1	1	1	1	1	1	1	3	3	3
Frame 2		2	2	2	2	2	2	2	2	2	4	4
Frame 3			3	4	4	4	5	5	5	5	5	5
Fault?	Y	Y	Y	Y	N	N	Y	N	N	Y	Y	N

Total Page Faults: 7 (with 3 frames)

3. LRU (Least Recently Used)

Replace the page that **has not been used for the longest time**. Good approximation of OPT. Implementation is complex (needs counters or stack).

Reference	1	2	3	4	1	2	5	1	2	3	4	5
Frame 1	1	1	1	4	4	4	5	5	5	3	3	3
Frame 2		2	2	2	1	1	1	1	1	1	4	4
Frame 3			3	3	3	2	2	2	2	2	2	5
Fault?	Y	Y	Y	Y	Y	Y	Y	N	N	Y	Y	Y

Total Page Faults: 9 (with 3 frames)

4. NRU (Not Recently Used)

Pages are classified into four classes based on reference bit (R) and modified bit (M):

- **Class 0:** R=0, M=0 — Best to replace
- **Class 1:** R=0, M=1
- **Class 2:** R=1, M=0
- **Class 3:** R=1, M=1 — Worst to replace

Replace a page from the lowest-numbered non-empty class.

5. Second Chance (Clock) Algorithm

Modified FIFO with a **reference bit**. Pages are arranged in a circular list. A pointer cycles through frames:

- If reference bit = 1, set it to 0 and move on (give second chance)
- If reference bit = 0, replace this page

Practical and efficient.

6. LFU (Least Frequently Used)

Replace the page with the **smallest reference count**. If multiple pages have the same count, use LRU among them.

Thrashing

Thrashing is a condition where the system spends most of its time **swapping pages** between memory and disk rather than executing processes. The page fault rate is very high, and CPU utilization drops drastically.

Causes of Thrashing

- Too few frames allocated to a process
- High degree of multiprogramming
- Page replacement algorithm not working well

Working Set Model

Working Set = Set of pages a process is currently using
Working Set Size = Number of pages in the working set

If the sum of all working sets exceeds available memory → Thrashing occurs.
Solution: Ensure each process has enough frames for its working set.

Page Fault Frequency (PFF) Control

- If page fault rate is too high → allocate more frames to the process
- If page fault rate is too low → remove frames from the process
- If total working set > memory → reduce degree of multiprogramming (suspend a process)

Important Formulas

Concept	Formula
Physical Address	Frame Number × Frame Size + Offset
Page Number	Logical Address / Page Size
Offset	Logical Address % Page Size
Number of Pages	Program Size / Page Size
Number of Frames	Physical Memory Size / Frame Size
Page Table Size	Number of Pages × Page Table Entry Size
Effective Access Time	(1-p) × Memory Access Time + p × Page Fault Time
Memory Utilization	(Used Memory / Total Memory) × 100%

Quick Revision Points

Topic	Key Points
Paging	Fixed blocks, frames and pages, no external fragmentation
Segmentation	Logical segments, external fragmentation possible
Virtual Memory	Illusion of large memory, demand paging
Demand Paging	Load pages on demand, page faults
Page Replacement	FIFO, LRU, OPT, Second Chance, NRU, LFU
Fragmentation	Internal (inside block), External (between blocks)
Swapping	Moving processes between memory and disk
Thrashing	High page fault rate, CPU spends time swapping

Compaction	Solve external fragmentation by moving processes

Questions

SECTION A: Very Short Answer Questions (2-3 Marks)

Q1. What is Memory Management? What are its functions?

Answer: Memory Management is the OS function that manages RAM. **Functions:** Allocating/deallocating memory, protecting memory from unauthorized access, managing virtual memory and swapping, efficiently utilizing available memory.

Q2. Define Paging and Segmentation.

Answer: Paging divides physical memory into fixed-size frames and logical memory into same-size pages. **Segmentation** divides a program into logical segments (code, data, stack, heap) based on program structure.

Q3. What is Fragmentation? Name its types.

Answer: Fragmentation is wasted memory due to small unusable chunks. **Types:** 1. Internal (unused space inside partition), 2. External (free memory in small scattered chunks).

Q4. What is Virtual Memory?

Answer: Virtual Memory creates an illusion of very large memory by using disk as extension of RAM, allowing programs larger than physical memory to run.

Q5. What is Demand Paging?

Answer: Pages are loaded into memory only when needed (on demand). When a process accesses a page not in memory, a page fault occurs and the page is loaded from disk.

Q6. Define Page Fault. How is it handled?

Answer: A Page Fault occurs when a process tries to access a page not in memory. **Handling:** 1. Suspend process, 2. Find free frame, 3. If none, select victim, 4. Load page from disk, 5. Update page table, 6. Resume process.

Q7. What is Thrashing?

Answer: Thrashing is when the system spends most time swapping pages rather than executing processes. Page fault rate is very high, CPU utilization drops drastically.

Q8. Differentiate between Internal and External Fragmentation.

Internal Fragmentation	External Fragmentation
Unused space inside a partition	Free memory in small chunks scattered
Occurs in Fixed Partitioning	Occurs in Dynamic Partitioning
Waste is within allocated memory	Waste is between allocated blocks
Cannot be used by any process	Total free memory exists but unusable

Q9. What is Swapping?

Answer: Swapping moves processes temporarily between main memory and secondary storage. Used to support multiprogramming, efficiently utilize memory, and allow more processes to run.

Q10. What is Compaction?

Answer: Compaction moves all allocated memory blocks together to create a single large free block. Used to solve external fragmentation. Expensive operation.

SECTION B: Short Answer Questions (5 Marks)

Q11. Explain Paging in detail with a neat diagram.

Answer: Paging divides physical memory into fixed-size frames and logical memory into pages. A page table maps page numbers to frame numbers. **Address Translation:** Physical Address = Frame Number × Frame Size + Offset. **Advantages:** No external fragmentation, efficient memory use. **Disadvantages:** Internal fragmentation possible, page table overhead.

Q12. Explain Segmentation in detail with a neat diagram.

Answer: Segmentation divides a program into logical segments. A segment table stores base address and limit for each segment. **Address Translation:** Physical

Address = Base + Offset (if Offset < Limit). **Advantages:** Logical grouping, per-segment protection. **Disadvantages:** External fragmentation.

Q13. Explain different types of Memory Management Schemes.

Scheme	Description	Advantages	Disadvantages
Bare Machine	No OS, one program at a time	Simple, Fast	No protection, inefficient
Resident Monitor	Small OS in memory	Automated job control	No multiprogramming
Swapping	Processes moved between memory/disk	More processes	Slow, high overhead
Fixed Partition	Memory divided into fixed partitions	Simple	Internal fragmentation
Dynamic Partition	Partitions created on demand	Efficient	External fragmentation
Paging	Fixed-size pages and frames	No external fragmentation	Internal fragmentation
Segmentation	Logical segments	Protection, sharing	External fragmentation
Virtual Memory	Illusion of large memory	Run large programs	Page faults, overhead

Q14. Explain FIFO, LRU, and Optimal Page Replacement Algorithms.

Answer: FIFO: Replaces oldest page; simple but has Belady's Anomaly. **LRU:** Replaces least recently used page; good approximation of OPT but complex. **Optimal:** Replaces page not used for longest future time; theoretical optimum, cannot be implemented.

Q15. Explain Virtual Memory and Demand Paging.

Answer: Virtual Memory uses disk as RAM extension. Demand Paging loads pages only when needed. **Benefits:** Programs larger than memory can run, more processes loaded, efficient memory utilization, faster program loading.

SECTION C: Long Answer Questions (10-15 Marks)

Q16. Explain Memory Management techniques: Paging and Segmentation in detail, comparing both.

Answer: Paging: Fixed size blocks, no external fragmentation, internal fragmentation possible, simple hardware. **Segmentation:** Variable size blocks, external fragmentation possible, no internal fragmentation, complex hardware. **Comparison:** Paging has fixed block size, internal fragmentation, linear user view, simple hardware. Segmentation has variable block size, external fragmentation, logical user view, complex hardware.

Q17. Explain Virtual Memory including Page Replacement Algorithms.

Answer: Virtual Memory uses disk as RAM extension. **Page Replacement Algorithms:** FIFO, OPT, LRU, Second Chance, NRU, LFU. **Thrashing:** High page fault rate where CPU spends time swapping. **Solutions:** Working Set Model, Page Fault Frequency control.

Q18. Explain various types of Fragmentation and their solutions.

Answer: Internal: Unused space inside allocated block (fixed partitioning, paging). **External:** Free memory in small chunks (dynamic partitioning). **Solutions:** Compaction (move processes together), Paging (eliminates external fragmentation).

End of UNIT - III: Memory Management